Empirical Circuit Complexity: Formal Proofs for Small Boolean Functions

Samuel Schlesinger

Abstract

We describe a research program to determine the exact circuit complexity of every Boolean function $\{0,1\}^n \to \{0,1\}$ for small n, with all results formally verified in Lean 4. Our circuit model uses fan-in 2 AND gates with free negation on wires. We exploit NPN equivalence—input permutation, input negation, output negation—to reduce the proof burden by orders of magnitude: from 65,536 functions at $n = 4$ down to 222 representatives. The proofs are written by AI within a trust architecture where Lean's type checker is the sole arbiter of correctness.

1 Introduction

Circuit complexity—the minimum number of gates needed to compute a Boolean function—is among the most fundamental measures in computational complexity theory. Despite decades of effort, proving superlinear circuit lower bounds for explicit functions remains one of the great open problems [1, 2].

The asymptotic theory has produced deep structural results: Lupanov's upper bound showing almost all functions require $\sim 2^n/n$ gates [3], and the Shannon counting argument [4] showing most functions are hard. For small n, exact circuit complexities have been determined computationally, notably in Knuth's exhaustive treatment [5]. But these results are computational artifacts, not proofs: they are outputs of programs that could, in principle, contain bugs.

This paper describes a program to produce *formal proofs* of the exact circuit complexity of every Boolean function on a small number of inputs. For n input bits, there are 2^{2^n} Boolean functions. The circuit complexity of each is a natural number, and we prove it correct—both by exhibiting an optimal circuit and by ruling out all smaller ones—as a machine-checked theorem in Lean 4.

Three aspects are worth noting:

1. **Formal verification.** Every result is a theorem that Lean's type checker has accepted. If any claim is wrong, the project does not compile.

2. **Symmetry exploitation.** We formally prove that circuit complexity is invariant under the NPN equivalence group, reducing the number of independent proofs by up to 295× at $n = 4$.

3. **AI-assisted proof generation.** The proof terms are written by large language models within a trust architecture that confines the AI to supplying evidence. Incorrect proofs are rejected by the type checker. This enables scale without sacrificing rigor.

Our initial target is $n = 1$ through $n = 4$: 242 representative proofs covering all 65,812 Boolean functions. Beyond the database itself, we are interested in the structural questions it surfaces: what is the distribution of circuit complexities? How do optimal circuits and lower bound arguments change as n grows?

The source code is available at `https://github.com/SamuelSchlesinger/empirical-circuit-complexity`.

2 Circuit Model

Definition 1. *A* Boolean circuit *with n inputs and k gates is a sequence of k fan-in 2 AND gates in topological order, together with a designated output wire. Each wire reference (gate input or circuit output) consists of a wire index and a* negation bit*; when the negation bit is set, the signal on the wire is inverted. The* size *of the circuit is k, the number of AND gates.*

Definition 2. *The* circuit complexity *of a Boolean function $f : \{0,1\}^n \to \{0,1\}$, written $\mathsf{C}(f)$, is the minimum k such that some circuit of size k computes f.*

Formally, a circuit is a pair $(\mathbf{g}, r)$ where $\mathbf{g}$ is a `GateList`—an inductively defined sequence of gates, each referencing only earlier wires—and r is a `Ref` (a wire index with negation bit) designating the output. Gate i can reference any of the $n + i$ available wires: n inputs plus i preceding gate outputs.

The formalization is self-contained: it uses Lean 4 with no external dependencies (no Mathlib or other libraries). This is a deliberate choice to keep the trusted base minimal, though we may reconsider as the proofs grow more complex.

Why this basis. The AND/NOT basis is functionally complete. Treating NOT as a zero-cost wire annotation reflects the standard convention that negation is "free" [2]. This model is equivalent to counting {AND, OR} gates when both are available, since $\mathrm{OR}(a, b) = \neg\mathrm{AND}(\neg a, \neg b)$ by De Morgan's law.

Constants cost one gate. There are no free constant wires. The cheapest way to produce a constant output is $\mathrm{AND}(x, \neg x) = \mathsf{false}$ (one gate), optionally negated for true.

3 Trust Architecture

A central design goal is a sharp trust boundary between *what is claimed* and *who generates the evidence*. For each input size n, three kinds of files are produced:

1. **Definitions** (`Defs`): Generated by a human-reviewed Python program. Each Boolean function is encoded as a Lean 4 function `(Fin` n `→ Bool) → Bool` via a nested if-then-else tree matching its truth table.

2. **Proofs**: Written by AI. Contains proof terms: upper bounds (exhibiting circuits) and lower bounds (ruling out smaller circuits). These are the only files not produced by the trusted generator.

3. **Statements**: Generated by the same trusted program. Imports both Definitions and Proofs and states the circuit complexity theorems, drawing proof terms from the Proofs module.

The trust model is: we trust the generator (which we review) and Lean's type checker. The AI is *untrusted*—it may hallucinate circuits, miscalculate complexities, or produce invalid proof terms. Any such error causes the Statements file to fail type-checking. Lean's module system enforces this boundary: the Statements file imports the Proofs file and requires that the exported terms have the correct types.

For each representative function, the proof file must export:

- `f_{idx}_size : Nat` — the claimed circuit complexity k,
- `f_{idx}_upper : HasCircuitOfSize f` k — a witness circuit of size k,
- `f_{idx}_lower : ∀ j, j <` k `→ ¬ HasCircuitOfSize f` j — a proof that no smaller circuit suffices.

4 Symmetry Exploitation

Several transformations on circuits are *free*: they change which function is computed without changing the number of gates. We identify three such transformations and formally prove that each preserves circuit complexity.

4.1 Three free transformations

Write f^σ for the function $x \mapsto f(x_{\sigma(0)}, \ldots, x_{\sigma(n-1)})$, i.e., f with inputs permuted by σ. Write $f^{\oplus i}$ for the function obtained by negating input i: $x \mapsto f(x_0, \ldots, \neg x_i, \ldots, x_{n-1})$.

Theorem 3 (Input permutation invariance)**.** *For any $\sigma \in S_n$, $\mathsf{C}(f^\sigma) = \mathsf{C}(f)$.*

Proof idea. Relabel each input wire reference $i \mapsto \sigma(i)$. No gates are added or removed.

Theorem 4 (Input negation invariance)**.** *For any input index i, $\mathsf{C}(f^{\oplus i}) = \mathsf{C}(f)$.*

Proof idea. Flip the negation bit on every reference to input wire i. No gates are added or removed.

Theorem 5 (Output negation invariance)**.** $\mathsf{C}(\neg f) = \mathsf{C}(f)$.

Proof idea. Flip the negation bit on the output wire. No gates are added or removed.

All three theorems and their converses are stated and proved in Lean 4 in our `Circuits/Basic.lean` module. The output negation proof is complete. The input permutation and input negation proofs depend on evaluation lemmas (showing that the modified circuit evaluates to the expected function) whose inductive proofs over the dependent gate list type are stated but deferred; the complexity invariance theorems are proved assuming these lemmas.

4.2 NPN equivalence

These three operations generate a group known in logic synthesis as the *NPN equivalence group* [6]:

$$G = (S_n \ltimes \mathbb{Z}_2^n) \times \mathbb{Z}_2,$$

of order $2^{n+1} \cdot n!$, where S_n acts on $\mathbb{Z}_2^n$ by permuting coordinates. Two functions are NPN-equivalent if and only if one can be obtained from the other by permuting inputs, negating inputs, and negating the output. The NPN class counts form OEIS sequence A000370 [7].

Remark 6. *NPN equivalence is the maximal equivalence relation on Boolean functions preservable by free wire operations in this model. The argument is straightforward: the only operations that preserve gate count are changes to wire indices and negation bits. On input wires, index changes yield S_n and negation bit changes yield $\mathbb{Z}_2^n$; on the output wire, negation bit changes yield the final $\mathbb{Z}_2$ factor. Any other input transformation—for instance, replacing x_i with $x_i \oplus x_j$—requires computing a new function of two wires, which costs at least one gate.*

4.3 Counting representatives via Burnside's lemma

The number of NPN equivalence classes is computed by Burnside's lemma:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\mathrm{Fix}(g)|.$$

Each group element $g = (\sigma, \varepsilon, \delta)$ induces a permutation π on the 2^n input indices: $\pi(j)$ is obtained by permuting the bits of j according to σ and then XORing with ε. A truth table is fixed by g if and only if $f(j) = f(\pi(j)) \oplus \delta$ for every input index j.

When $\delta = 0$, the truth table must be constant on each cycle of π, giving $2^{c(\pi)}$ fixed points where $c(\pi)$ is the number of cycles. When $\delta = 1$, values must alternate around each cycle: $f(j) \neq f(\pi(j))$. Following this constraint around a cycle of length L, we find $f(j) = f(\pi^L(j)) \oplus (L \bmod 2) = f(j) \oplus (L \bmod 2)$. If L is odd this forces $f(j) \neq f(j)$, a contradiction, so all cycles must have even length; when they do, there are $2^{c(\pi)}$ fixed points.

n	Functions	$\lvert G\rvert$	Representatives	Reduction
1	4	4	2	2×
2	16	16	4	4×
3	256	96	14	18×
4	65,536	768	222	295×
5	$\sim 4.3 \times 10^9$	7,680	616,126	6,971×
6	$\sim 1.8 \times 10^{19}$	92,160	$\sim 2.0 \times 10^{14}$	92,117×
7	$\sim 3.4 \times 10^{38}$	1,290,240	$\sim 2.6 \times 10^{32}$	1,290,240×

Table 1: NPN equivalence class counts. For large n, the reduction factor approaches $|G|$, indicating almost all functions have trivial stabilizer.

At $n = 4$, proving complexity for 222 representatives covers all 65,536 functions. For $n \leq 4$, only $2 + 4 + 14 + 222 = 242$ independent proofs are needed.

5 Proof Techniques

Two kinds of proofs are required for each function: an upper bound (exhibit a circuit) and a lower bound (rule out all smaller circuits).

5.1 Upper bounds

An upper bound constructs a concrete circuit and proves it computes the target function. The proof obligation is $\forall\, x,\ \texttt{eval}(c, x) = f(x)$. Because both the circuit and the function definition

are fully concrete, Lean's `simp` tactic—given the evaluation definitions—reduces both sides to identical Boolean expressions and closes the goal automatically.

5.2 Lower bounds

Lower bound proofs show that no circuit of size less than k computes f. This requires universal quantification over all circuits of each smaller size. Several patterns emerge at small n:

Vacuous. When $\mathsf{C}(f) = 0$, the lower bound $\forall\, j < 0, \ldots$ is vacuously true.

Ruling out size 0. A size-0 circuit is just a (possibly negated) input wire. A general lemma handles all constant functions: evaluating on the all-true and all-false inputs, a size-0 circuit must return different values, but a constant function returns the same value. Contradiction. For non-constant, non-literal functions, the proof case-splits on which input wire and negation bit, providing a counterexample input for each case.

Ruling out size ≥ 1. The proof must enumerate all possible gate configurations. For a single gate, there are $(2n)^2$ choices for the gate inputs (each of n wires with a negation bit, two inputs) and $2(n+1)$ choices for the output reference. The proof destructures the circuit, case-splits on all choices, and provides counterexample inputs. For size ≥ 2, the enumeration grows combinatorially.

5.3 Example: constant false on one input

To illustrate the proof structure, consider f_0 on $n = 1$: the constant-false function. Its complexity is 1.

Upper bound. We exhibit the circuit $\mathrm{AND}(x_0, \neg x_0)$ with output referencing the gate (non-negated):

```
def f_0_upper : HasCircuitOfSize f_0 1 :=
  ({gates := .cons .nil {lhs := {index := (0 : Fin 2),
                                 negated := false},
                         rhs := {index := (0 : Fin 2),
                                 negated := true}},
    output := {index := (1 : Fin 2), negated := false}},
   fun input => by
     simp [Circuit.eval, Ref.eval, GateList.eval,
           Gate.eval, extendEnv, f_0])
```

The anonymous constructor provides the circuit term; `simp` verifies that evaluating it on any input produces false, matching f_0.

Lower bound. We must show no size-0 circuit computes f_0. Since $f_0(\mathbf{1}) = f_0(\mathbf{0}) = \mathsf{false}$, the general constant lemma applies directly:

```
def f_0_lower :
    forall j, j < 1 -> Not (HasCircuitOfSize f_0 j) := by
  intro j hj
  have : j = 0 := by omega
```

```
  subst this
  exact no_size0_of_constant (by simp [f_0])
```

The `omega` tactic establishes $j = 0$; then the general lemma rules out size 0.

Remark 7. *A recurring implementation concern: Lean's* `simp` *tactic diverges when applied to circuit evaluation terms containing free Boolean variables (e.g., an unresolved negation bit). The workaround is to case-split on all Boolean parameters* before *invoking* `simp`*.*

6 Results

6.1 Completed: $n = 1$

All 4 functions (2 NPN representatives) have been formally verified:

Index	Function	C	Witness
0	constant 0	1	$\mathrm{AND}(x_0, \neg x_0)$
1	$\neg x_0$	0	wire with negation

The two non-representatives are x_0 (output negation of $\neg x_0$) and constant 1 (output negation of constant 0).

6.2 Expected: $n = 2$

The 16 functions reduce to 4 NPN representatives. Formal proofs are in progress; the expected complexities (matching Knuth's computed values [5]) are:

Index	Function	C (expected)	Witness
0	constant 0	1	$\mathrm{AND}(x_0, \neg x_0)$
1	$\neg x_0 \wedge \neg x_1$	1	$\mathrm{AND}(\neg x_0, \neg x_1)$
3	$\neg x_1$	0	wire with negation
6	$x_0 \oplus x_1$	3	see below

The XOR function is notable: it requires 3 AND gates, making it the most expensive 2-input function. One optimal circuit computes it as $\neg(\neg(x_0 \wedge \neg x_1) \wedge \neg(\neg x_0 \wedge x_1))$: the two inner gates compute the "half-XORs" $x_0 \wedge \neg x_1$ and $\neg x_0 \wedge x_1$, and the outer gate takes their NOR and negates. The lower bound proof—ruling out all circuits of sizes 0, 1, and 2—will require significant case analysis.

6.3 Baseline observations

At $n = 1$, the results establish baseline behavior for the cost model:

- **Negation is free in practice.** The literal $\neg x_0$ has complexity 0. This confirms the model: NOT is a wire annotation, not a gate.
- **Constants are not free.** With no constant wires available, producing a constant costs one gate ($x \wedge \neg x$). This is a consequence of the model choice and sets the floor for all non-literal functions.

- **NPN reduction is effective.** Even at $n = 1$, the four functions collapse to two classes. At $n = 2$, 16 functions collapse to 4 classes. At $n = 4$, the 295× reduction makes the project feasible.

These are not surprises—they follow directly from the definitions—but they validate that the formal model behaves as intended before we scale to larger n where the results are no longer obvious.

7 Related Work

Exact circuit complexity. Knuth [5] gives an extensive computational treatment of circuit complexity for small Boolean functions, including exhaustive search for optimal circuits and tables of exact values. Our work produces the same values but as formally verified theorems rather than computational outputs. The formal proofs also serve a different purpose: they are explicit, inspectable objects from which proof strategies can be extracted and analyzed.

NPN classification. NPN equivalence is a standard tool in logic synthesis for reducing the space of Boolean functions [6]. The class counts (OEIS A000370 [7]) have been tabulated for n up to at least 6. We apply this classification not for synthesis but to reduce the number of formal proofs required.

Formal verification of combinatorics. The use of proof assistants for large combinatorial results has precedent: the Flyspeck project [8] (Kepler conjecture), the Four Color Theorem formalization [9], and various results in Lean's Mathlib [10]. Our work is distinguished by its focus on circuit complexity and its use of AI-generated proof terms within a minimal trusted base.

AI-assisted theorem proving. Recent work has shown the potential of large language models for formal proof generation [11]. Our trust architecture is designed to exploit this: the AI generates candidate proof terms, and the type checker accepts or rejects them. The human role is limited to reviewing the trusted generator and evaluating the proof strategies that emerge.

8 Open Questions

The systematic, verified nature of this effort surfaces several concrete questions that we intend to investigate as the database grows.

Complexity distribution. What is the distribution of $\mathsf{C}(f)$ over NPN representatives at each n? How does the maximum complexity grow? The Lupanov bound gives $\max_f \mathsf{C}(f) \leq (1 + o(1)) \cdot 2^n/n$ asymptotically [3]; our data will provide exact values for small n.

Structure of hard functions. Which NPN classes have the highest complexity? Do they share structural features? XOR is the hardest function at $n = 2$; what are its counterparts at $n = 3$ and $n = 4$?

Lower bound composition. Can lower bound proofs be composed inductively across levels of n? Shannon decomposition—fixing one variable and reducing to two $(n-1)$-input subfunctions—is a natural strategy. Our formal framework supports cross-size imports, enabling proofs at level n to cite verified results from level $n-1$.

Automation frontier. At $n = 4$ (222 representatives), AI-assisted proof generation is feasible with per-function interaction. At $n = 5$ (616,126 representatives), it requires genuine automation: batch proof search guided by patterns from smaller n. Upper bounds are likely automatable (search for circuits); lower bounds are the bottleneck.

Extracting proof structure. Every formal proof is an explicit term available for inspection. The counterexample inputs used in lower bound proofs, the circuit topologies that appear in optimal solutions, and the case-analysis patterns that close goals are all concrete data. Systematically mining this data may reveal structure invisible to the asymptotic theory.

9 Conclusion

We have described a research program to determine the exact circuit complexity of all Boolean functions for small n, with formal verification in Lean 4 as the standard of correctness. The NPN symmetry group reduces the proof burden by orders of magnitude, from 65,812 functions to 242 representatives for $n \leq 4$. An AI-assisted workflow generates proof candidates that the type checker verifies independently, enabling scale without sacrificing rigor.

The $n = 1$ case is complete; $n = 2$ through $n = 4$ are in progress. As the verified database grows, we expect it to serve as a testbed for questions about circuit complexity—one where every claimed fact is machine-checked and every proof term is available for inspection.

References

[1] A. A. Razborov, S. Rudich, "Natural proofs," *J. Comput. Syst. Sci.*, 55(1):24–35, 1997.

[2] S. Arora, B. Barak, *Computational Complexity: A Modern Approach*, Cambridge University Press, 2009.

[3] O. B. Lupanov, "On the synthesis of contact networks" (*O sinteze kontaktnykh skhem*), *Dokl. Akad. Nauk SSSR*, 119(1):23–26, 1958.

[4] C. E. Shannon, "The synthesis of two-terminal switching circuits," *Bell Syst. Tech. J.*, 28(1):59–98, 1949.

[5] D. E. Knuth, *The Art of Computer Programming*, Vol. 4A: *Combinatorial Algorithms, Part 1*, Addison-Wesley, 2011. See Section 7.1.2.

[6] D. Debnath, T. Sasao, "Efficient computation of canonical form for Boolean matching in large libraries," *Proc. ASP-DAC*, 2004.

[7] OEIS Foundation, "A000370: Number of NPN-equivalence classes of Boolean functions of n variables," `https://oeis.org/A000370`.

[8] T. Hales et al., "A formal proof of the Kepler conjecture," *Forum Math. Pi*, 5:e2, 2017.

[9] G. Gonthier, "Formal proof—the four-color theorem," *Notices AMS*, 55(11):1382–1393, 2008.

[10] The mathlib Community, "The Lean mathematical library," *Proc. CPP*, 2020.

[11] K. Yang et al., "LeanDojo: Theorem proving with retrieval-augmented language models and data extraction from Lean 4," *NeurIPS*, 2023.